Part 1: Development Environment Setup

Python Installation

Python is the main language used for this project. To install it, you download Python from the official Python website and run the installer. During installation, it's important to check the option that says **"Add Python to PATH."** This makes it easier to run Python commands from the terminal. After installation, Python can be checked by typing `python --version` in the command prompt.

TensorFlow and TensorFlow Lite Installation

TensorFlow is used to build and train the neural network, while TensorFlow Lite is used to prepare the model for edge devices. Both can be installed using pip. After Python is set up, the following commands are used:

```
pip install tensorflow
pip install tensorflow-lite
```

These commands download everything needed to work with both TensorFlow and TensorFlow Lite.

Jupyter Notebook Installation

Jupyter Notebook helps run and test machine learning code. It can be installed using:

```
pip install notebook
```

Once installed, it can be launched by typing `jupyter notebook`, which opens a browser window for creating and running notebooks.

Part 2: Conceptual Model Creation and Training

Neural Network Design

The MNIST dataset contains 28×28 grayscale images of handwritten digits from 0 to 9. A simple neural network can be built using Keras with the following structure:

- Input shape: 28×28
- Flatten layer to convert the image into a single vector
- Dense layer with 128 neurons using ReLU activation
- Dense layer with 64 neurons using ReLU activation
- Output layer with 10 neurons using Softmax activation

ReLU helps the model learn patterns, and Softmax is used to predict which digit is most likely.

Data Loading and Preprocessing

MNIST is loaded directly from Keras, which provides training and testing data. The pixel values range from 0 to 255, so the data is normalized by dividing by 255. This scales the values between 0 and 1, making training faster and more stable.

Model Compilation and Training

Before training, the model is compiled using:

- Adam optimizer
- Sparse Categorical Crossentropy loss function
- Accuracy as the evaluation metric

The model is then trained on the training data for about **5–10 epochs**, which is usually enough for MNIST.

Part 3: Model Conversion and Saving

TensorFlow Lite Conversion

After training, the model is converted to TensorFlow Lite format. This step reduces the model size and makes it more efficient for devices with limited resources. The TensorFlow Lite converter is used to transform the Keras model into a `.tflite` file.

Saving the Model

The converted TensorFlow Lite model is saved as a `.tflite` file. This file can later be loaded onto an edge device or used for testing.

Part 4: Conceptual Deployment and Testing

To deploy the model, a script loads the `.tflite` file using the TensorFlow Lite interpreter. The interpreter allocates tensors, takes in a sample image, and runs inference. The output is a list of probabilities for each digit, and the digit with the highest value is chosen as the prediction.

Reflective Journal

Introduction

This reflective journal focuses on my experience researching and conceptually designing an AI model using Python, TensorFlow, Keras, and TensorFlow Lite. The assignment walked through the full process of setting up a development environment, designing and training a neural network, converting the model to TensorFlow Lite, and conceptually deploying it on an edge device. The purpose of this reflection is to discuss what I learned, the challenges I faced, and how this experience helped me better understand how AI models are developed and used in real-world situations.

Description of the Experience or Topic

This assignment was different from others because it focused more on understanding the *process* of AI development rather than writing code. I had to research how Python and machine learning tools are installed, how a neural network is designed for the MNIST dataset, and how trained models are prepared for deployment using TensorFlow Lite. Instead of running experiments, I documented each step conceptually and explained why each part of the process matters.

The main focus was understanding how an AI model moves from training on a computer to being deployed on an edge device with limited resources. This helped connect topics we have covered in class, such as neural networks and optimization, to real-world applications.

Personal Reflection

At the beginning of this assignment, I felt a little overwhelmed because there were a lot of tools involved, and each one had a different purpose. Python, TensorFlow, TensorFlow Lite, and Jupyter Notebook all sounded familiar, but I hadn't really thought about how they fit together in a single workflow. The part that confused me the most was TensorFlow Lite, since it focuses on deployment rather than training.

As I worked through the research, things started to make more sense. I realized that training a model is only one part of the process and that deployment is just as important, especially when working with edge devices. This assignment helped me better understand why preprocessing data, choosing the right model architecture, and optimizing models are so important.

This experience connected well with what we have learned in class about neural networks, activation functions, and model training. Seeing how these concepts are used outside of just theory made them feel more practical and easier to understand.

Discussion of Improvements and Learning

This assignment contributed to my academic growth by helping me see the bigger picture of AI development. I developed a stronger understanding of how models are prepared for real-world use and why lightweight frameworks like TensorFlow Lite are necessary. I also improved my ability to explain technical concepts clearly without relying on code, which is an important skill.

If I were to improve anything, I think actually deploying a TensorFlow Lite model on physical hardware would help reinforce these concepts even more. Hands-on experience would make the deployment process feel less abstract.

In the future, I can apply what I learned in projects that involve edge computing, mobile AI, or embedded systems. Understanding the full pipeline from model creation to deployment will be useful in both academic projects and real-world applications.

Conclusion

Overall, this assignment helped me better understand the complete lifecycle of an AI model. Instead of focusing only on training accuracy, I learned how important deployment and optimization are, especially for devices with limited resources. This reflection helped reinforce key AI concepts while also showing how they apply beyond the classroom.